

Introduction to the Unified Modeling Language (UML)

Part 1 – EXE - P2

Marcello Missiroli & Giancarlo Succi

Object Oriented Analysis

OOA- A Generic View

- extract candidate classes
- establish basic class relationships
- define a class hierarchy
- identify attributes for each class
- specify methods that service the attributes
- indicate how classes/objects are related
- build a behavioral model
- iterate on the first five steps



Analysis: how to proceed

- Extract a set of analysis classes from the problem specification (we will discuss this shortly).
- Reason about these classes: what attributes and operations should they provide?
- Draw a map of the classes and their relationships.
- Model the dynamics of the classes using behavior diagrams.
- Proceed through successive refinements until the model effectively represents the problem domain.

Analysis: how to start

- An **analysis class** (or initial class) models a concept or entity of the problem: if the use case specification is well written, the fundamental concepts are already evident.
- The most likely candidates are the nouns that appear in the specification and documentation.
- This is one more reason to maintain a project glossary: the terms in the glossary are often ideal candidates to become analysis classes.
- Analysis classes will not necessarily survive into the design phase.



Analysis: noun-verb analysis

- All available documentation is analyzed, selecting nouns and verbs.
- Nouns (e.g., bank account) are potential candidates to become classes or attributes.
- Nominal predicates (e.g., account number) are potential candidates to become classes or attributes.
- Verbs (e.g., open) are potential candidates to become class responsibilities, hinting possible association

Example: Pub

Extract the analysis classes for the following domain:

- A pub is populated by customers and by staff. In particular, the staff collect orders and deliver beers. Payment is made at the cash register (and the personnel can give change if necessary). The manager is responsible not only for service, but also for checking the availability of each beer in the fridge, and, if necessary, restocking it.



Example: Pub

Extract the analysis classes for the following domain:

- A **pub** is populated by **customers** and by **staff**. In particular, the staff collect **orders** and **deliver beers**. **Payment** is made at the **cash register** (and the **personnel** can give **change** if necessary). The **manager** is responsible not only for service, but also for **checking the availability** of each **beer** in the **fridge**, and, if necessary, restocking it.

Classes

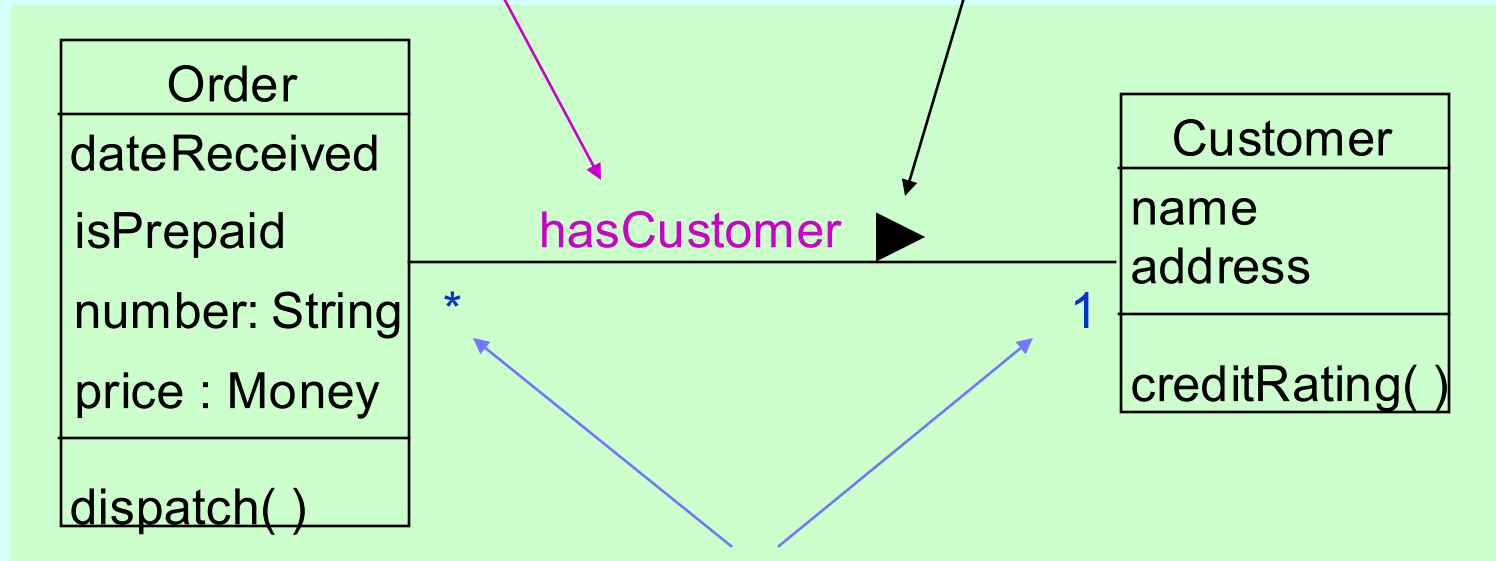
- Staff (handles clients)
- Manager(subclass of Staff, handles Fridge& Beers)
- Client
- Order
- Beer
- Fridge (holds beer)



Association: Relationship between classes

Name of the Association

Direction of the Association



Multiplicity of each end

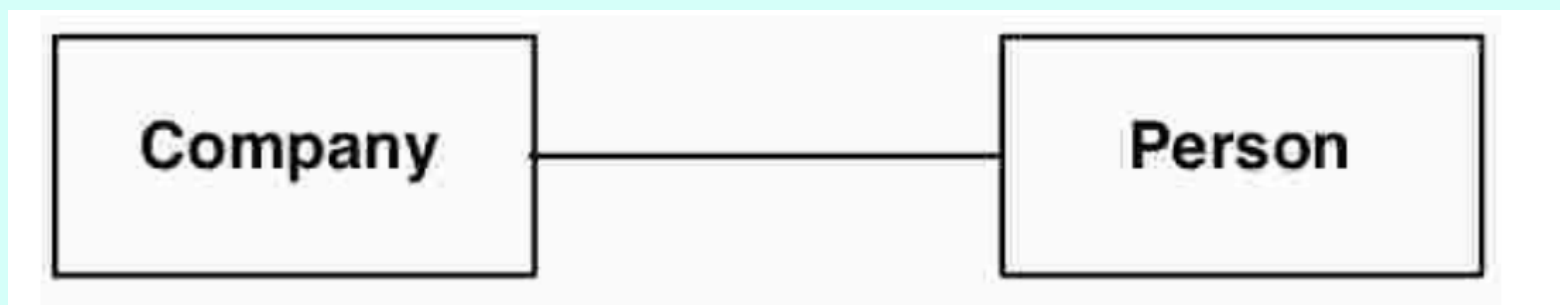
These elements are applicable to any relationship

Type of Association in UML

- Relationship between instances of classes
 - Association (generic)
 - Generalization
 - Realization
 - Aggregation
 - Composition

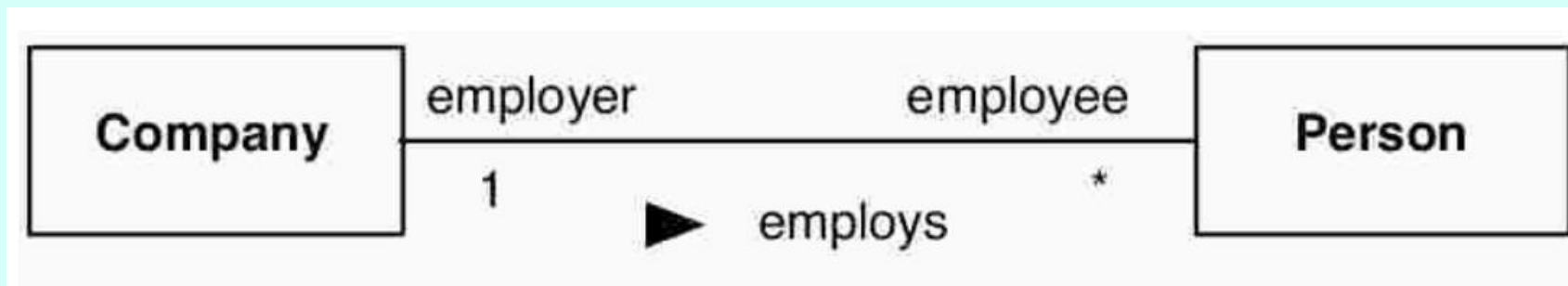
Association relationship

- This is the most generic type of relationship: it only indicates the existence of links between class instances.
- It represents the ability of one instance to send messages to another instance.
- It can involve more than two classes, and the same class more than once.
- Among relationships, it is also the most flexible and the least restrictive.



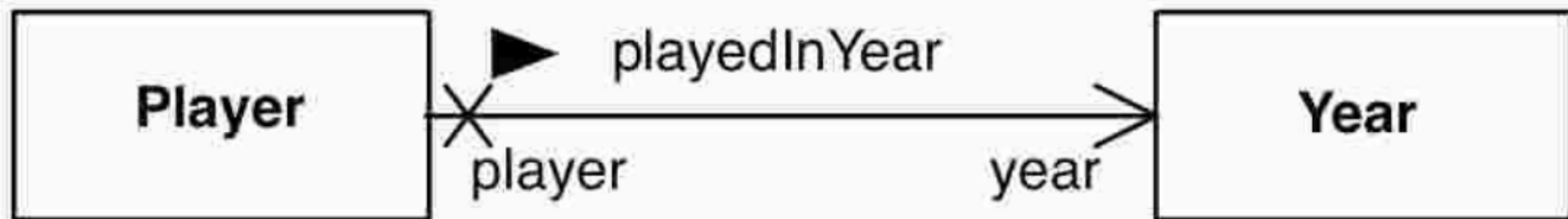
Association roles & directions

- Name: optional. BUT mandatory for associations between objects of the same class (reflexive associations)
- Directional triangle: optional. Specifies the direction in which to read the association (improves readability).
- Roles: optional at each end. Implicitly named after the target class.
- Multiplicity: optional at each end. Indicates how many object can participate in the relationship



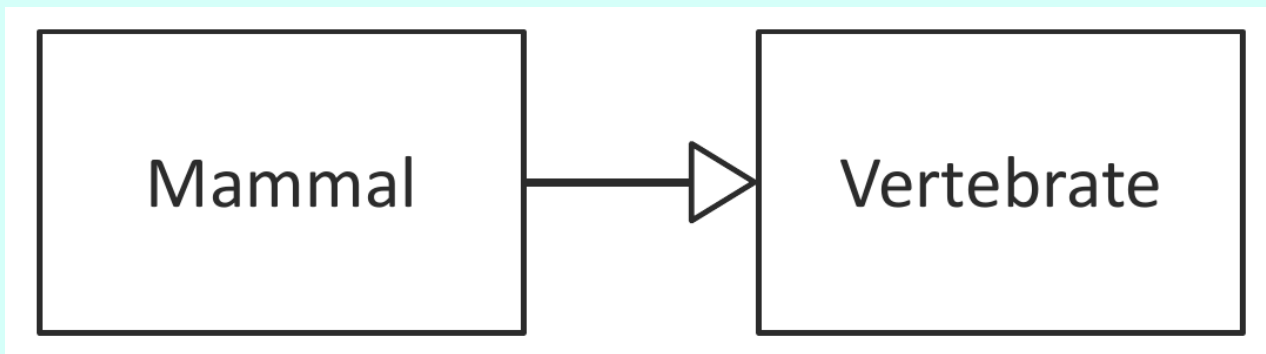
Association navigation

- Specifies whether the other ends of the association can know which instances they are associated with.
- The arrow indicates navigability.
- The cross indicates absence of navigability. (rare)
- The absence of both means navigability is unspecified (typical of the analysis phase).



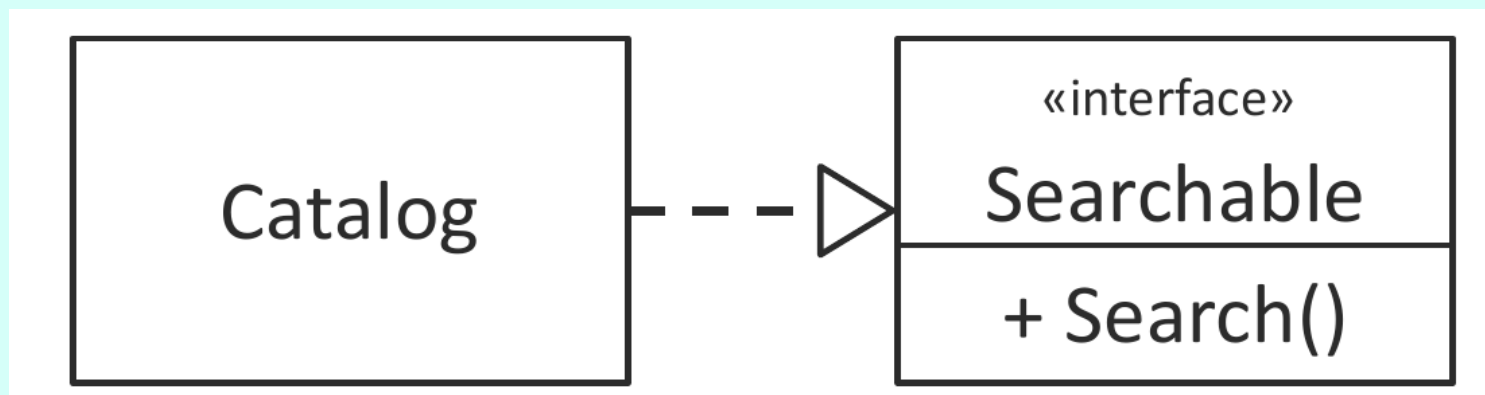
Generalization (recap)

- The arrow starts from the specific (child) element and points to the more general (parent) element.
- Among all UML relationships, this is the strongest and most constraining.



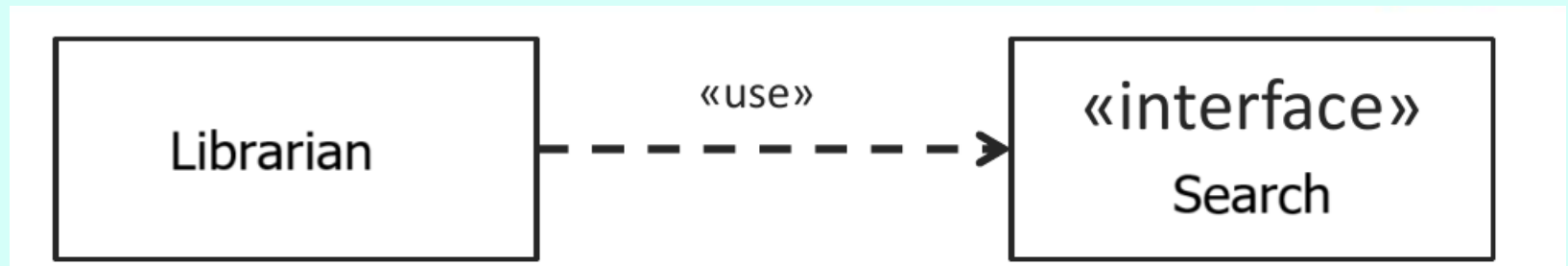
Realization (recap)

- A semantic relationship in which the supplier provides a specification, and the client realizes it (by implementing and executing it).
- The canonical example of a realization is when the supplier is a Java-style interface, and the client is the class that implements it.



Dependency

- A **dependency** is a weaker form of relationship between a client and a service provider (e.g., between classes and operations). There are two roles: supplier and client; both can be sets of elements.
- The arrow goes from the client to the supplier (and the type of dependency can be indicated).
- The client is structurally or semantically dependent on the supplier, and if the supplier's specification changes, the client's may also change.





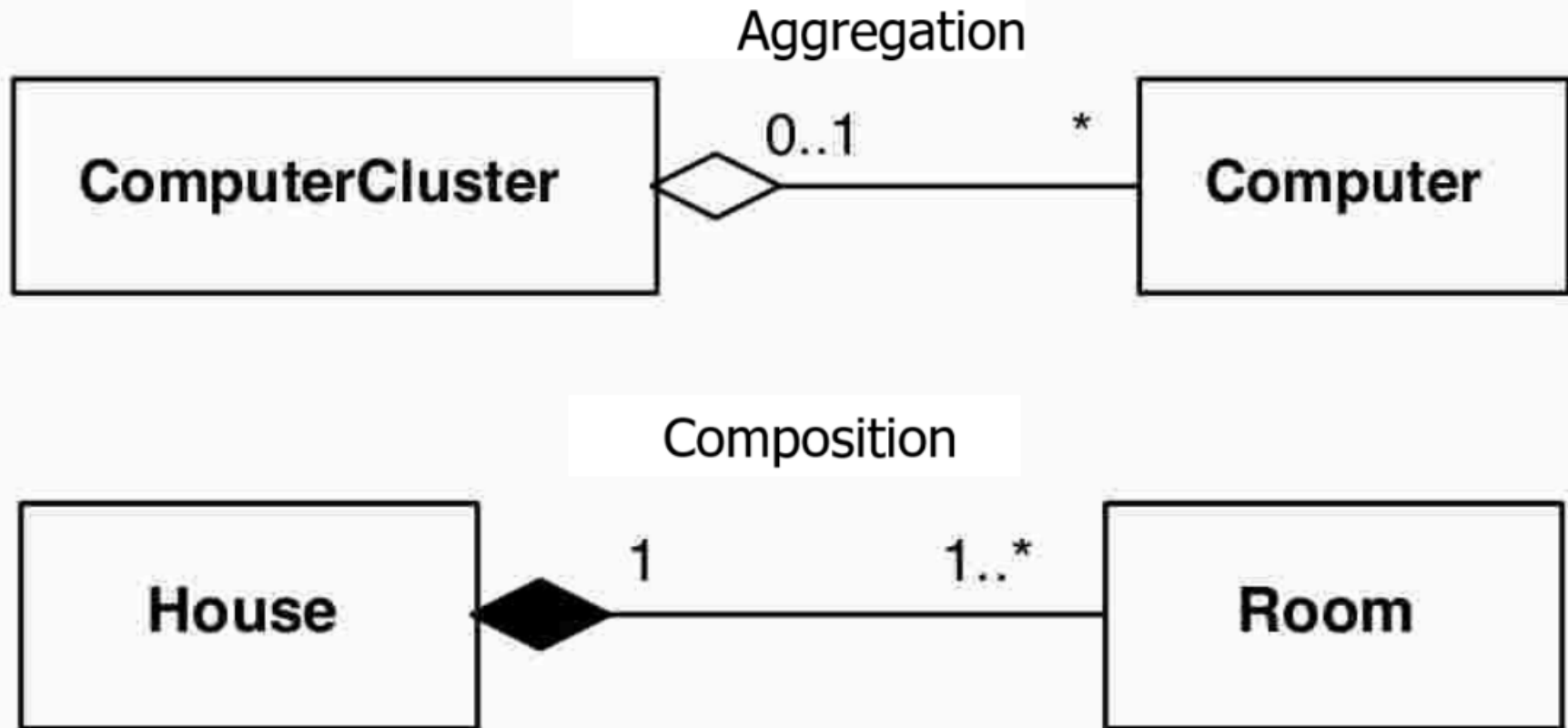
Aggregation & Composition

These are particular forms of association that represent the **whole-part** relationship between an aggregate and its parts.

- Aggregation: A less strong relationship. The parts can exist even without the whole, and they can belong to multiple aggregations.
 - Example: computers and their cluster.
- Composition: A very strong relationship. The parts are dependent on the whole and cannot exist outside of it.
 - Example: rooms and a house.

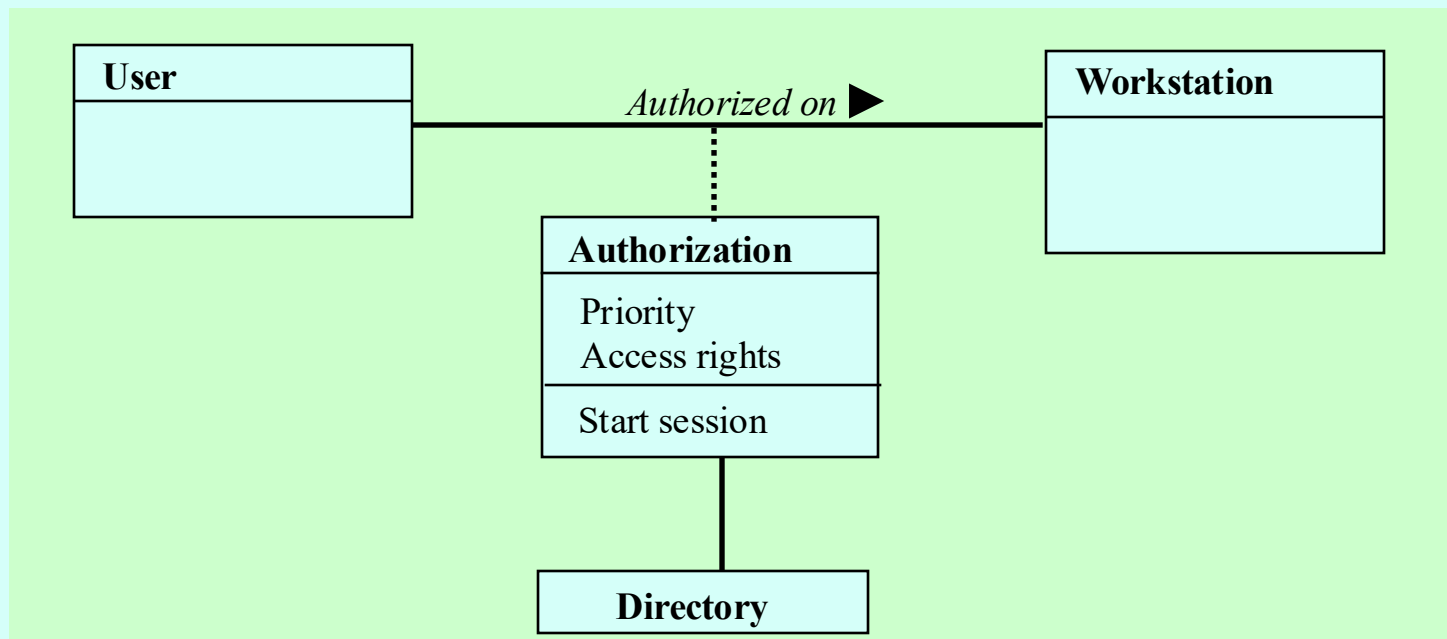
It's not always simple to figure out which of the two better models a situation.

Aggregation & Composition

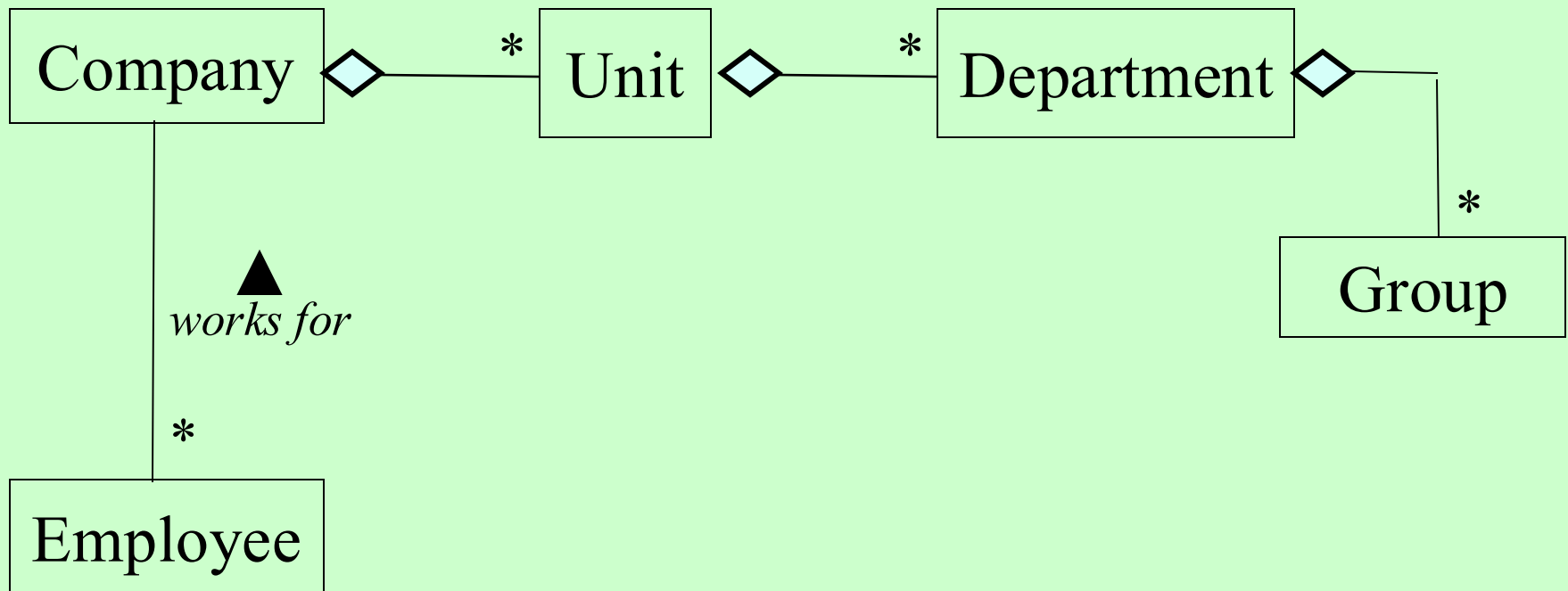


Association classes

- Useful if attributes don't belong to any one class but to the association (as in Relational Databases)



Example: Aggregation



Association or aggregation?

- Question: association or aggregation
 - Description “part of” correct?
 - Operation on whole affects parts too?

OOA Exercises

Creating a class diagram

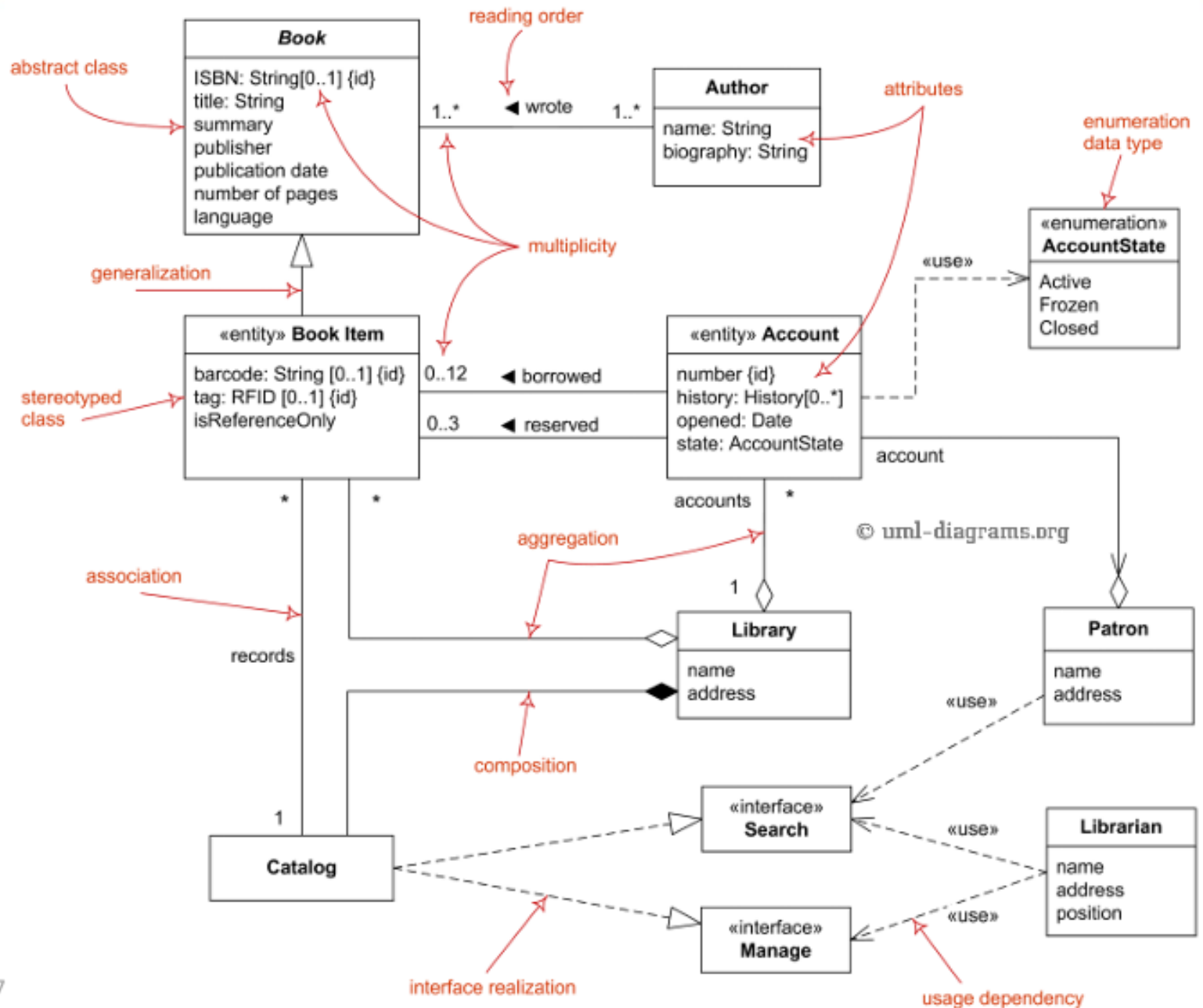
- Start simple
 - major classes & obvious associations
- Then add
 - Attributes
 - Multiplicity
 - Operations

Rules of thumb

- One class can be part of several diagrams
- Diagrams shall illustrate specific aspects
 - Not too many classes
 - Not too many associations
 - Hide irrelevant attributes/operations
 - Avoid “God Objects”
- Several iterations needed to create diagram



Full Example





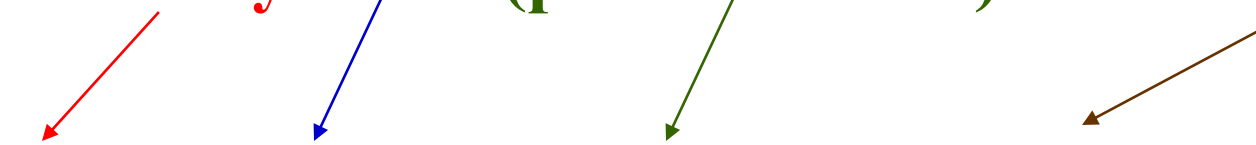
Association Cheat Sheet (it)

- Generalizzazione 
- Realizzazione 
- Associazione 
- Dipendenza 
- Aggregazione 
- Composizione 



Method syntax cheat sheet

visibility **name** (parameter list) : return-type-expression


+ assignAgent (a : Agent) : Boolean

visibility: **public** (+), **protected** (#), **private** (-)

Interpretation is language dependent

Nor needed on conceptual level

name: string

parameter list: arguments (syntax as in attributes)

return-type-expression: language-dependent specification

Exercise 4

Represent the following text via an UML Class diagram

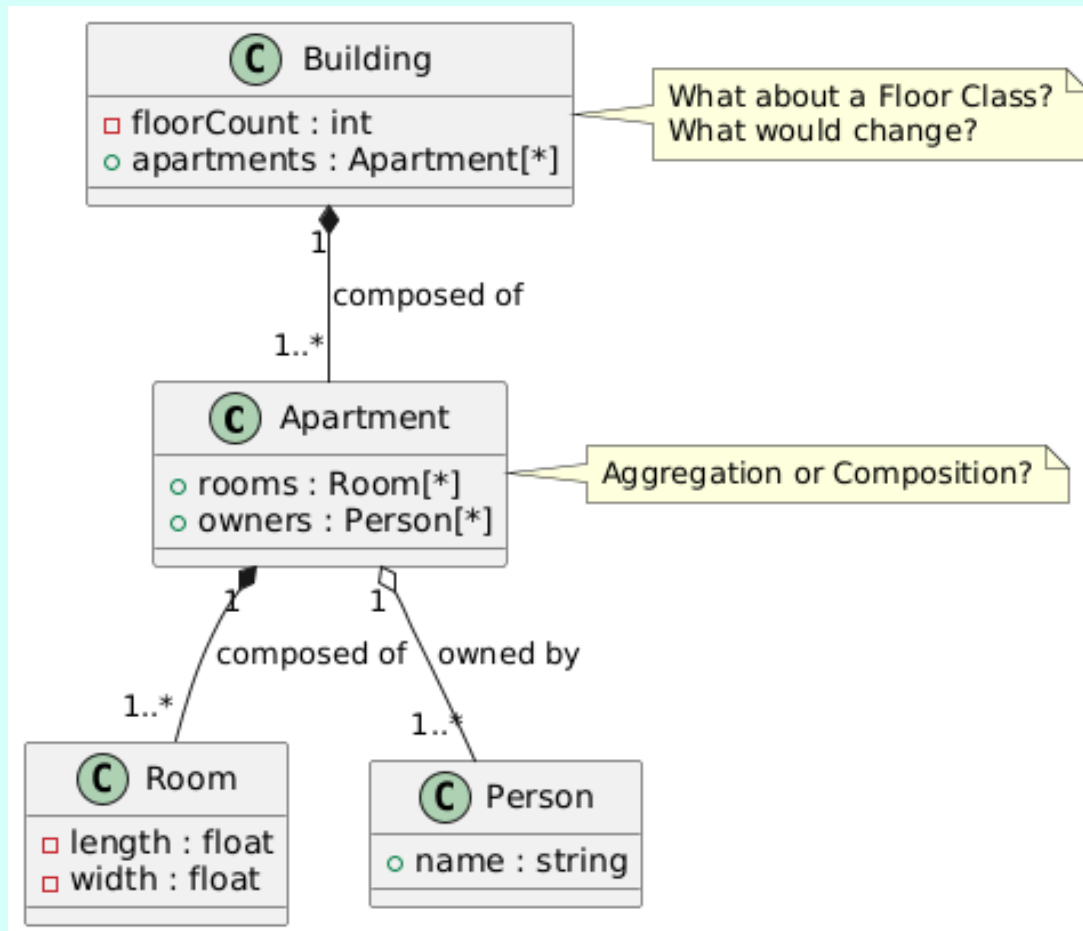
An apartment is composed of one or more rooms, each of which has a length and a width. An apartment is owned by one or more people. A building has a certain number of floors and is composed of one or more apartments.

Exercise 4

Represent the following text via an UML Class diagram

An **apartment** is composed of one or more **rooms**, each of which has a *length* and a *width*. An apartment is owned by one or more **people**. A **building** has a certain number of **floors** and is composed of one or more apartments.

Exercise 4 – solution



Exercise 5

Represent the following text via an UML Class diagram:

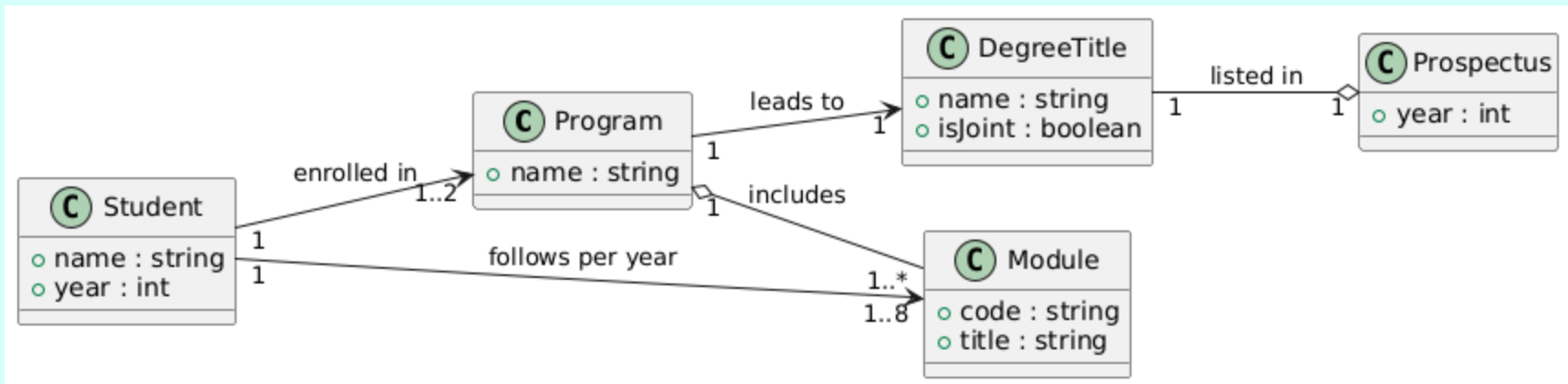
- De Montfort University (DMU) offers educational programs, each of which leads to the attainment of a degree title.
- Each degree title is advertised in DMU's information prospectus.
- Each program includes different modules.
- Students enrolled in a program take up to 8 modules per year.
- Some degree titles are joint; for example, a student can enroll in two different programs (such as accounting and finance).

Exercise 5

Represent the following text via an UML Class diagram:

- De Montfort University (DMU) offers educational **programs**, each of which *leads* to the *attainment* of a **degree** title.
- Each degree title is *advertised* in DMU's information **prospectus**.
- Each program *includes* different **modules**.
- Students *enrolled* in a program take up to 8 modules per year.
- Some degree titles are **joint**; for example, a student can enroll in two different programs (such as accounting and finance).

Exercise 5 Solution



Exercise 5a

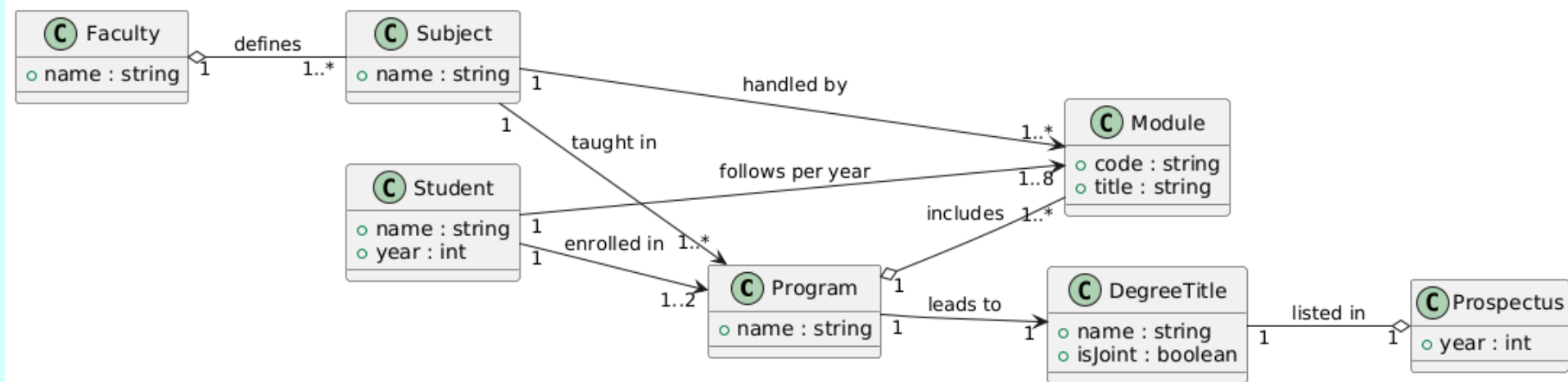
Complete the diagram to represent (Part 1):

- DMU is composed of 6 Faculties.
- Each Faculty defines a number of Subjects (e.g., Accounting, Auditing, etc.), which are handled by different Modules and are taught in different Programs.

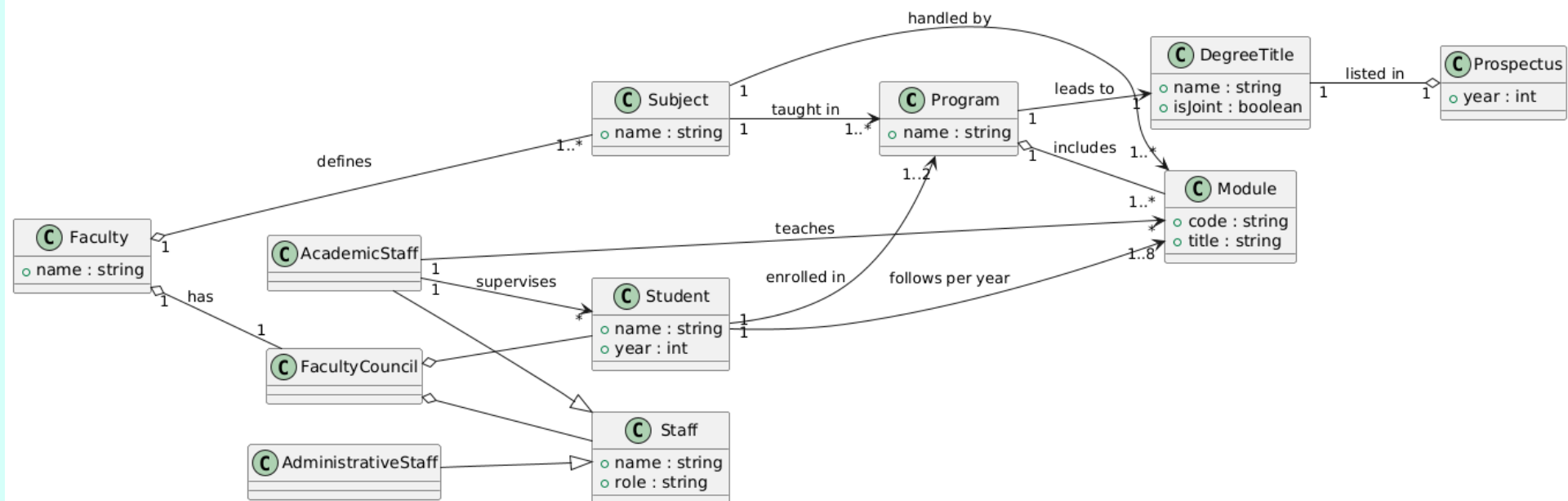
Then (Part 2):

- The Faculty Council is composed of Students and Academic or Administrative Staff.
- Academic Staff teach an arbitrary number of Modules.
- Academic Staff supervise several Students, each of whom is enrolled in a Program.
- Some Administrative Staff are Council Members but do not teach.

Exercise 5a part 1 solution



Exercise 5a full solution



Exercise 6 (proposed)

A nightclub management system allows for the organization of private parties. A party has a host (the person who sends the invitations), many guests, a DJ, some waiters, and a bouncer.

The system must allow:

- inviting people,
- receiving their responses (accept or decline),
- defining a bar menu of cocktails and drinks,
- creating a playlist.

The waiters serve the drinks, the DJ manages the playlist and also accepts requests during the party. The bouncer ensures that only invited guests are allowed into the party.

Draw a class diagram that models the domain described above.



Complex Exercise 1 solution

Stocks Trading Service

- By connecting to the service, a user can connect to different banks to acquire stock prices. The system also allows the user to perform some trend and prediction analysis of prices. If users are interested in ordering some stocks, they can choose to order them immediately or with a delay. They can also either bid at a single price or within a range of prices.
- The system should handle the situation where the connection to a bank is down, there is a conflict of bids, or if a particular stock is no longer available.



Complex Exercise 1

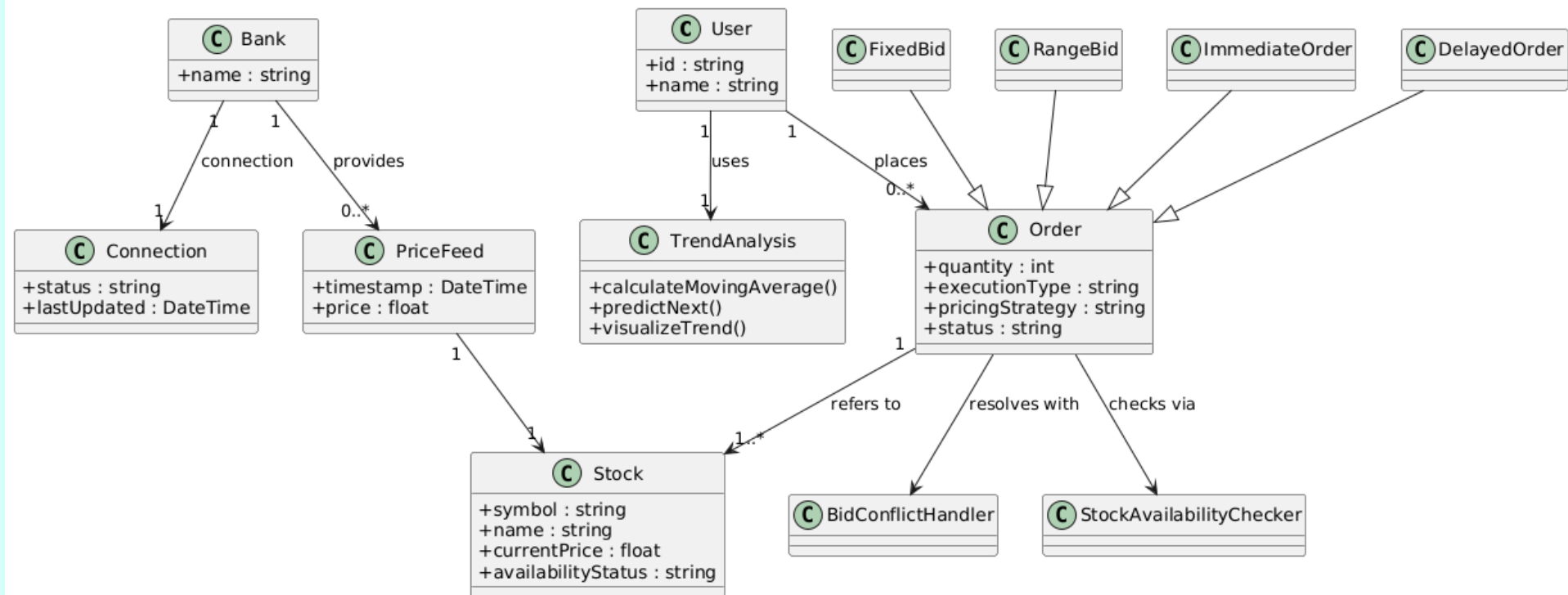
Stocks Trading Service

- By connecting to the service, a **user** can connect to different **banks** to acquire **stock** prices. The system also allows the user to perform some **trend and prediction analysis** of prices. If users are interested in **ordering** some stocks, they can choose to order them *immediately or with a delay*. They can also either **bid** at a *single price* or within a *range of prices*.
- The system should handle the situation where the connection to a bank *is down*, there is a *conflict* of bids, or if a particular stock is *no longer available*.



Complex Exercise 1 solution

Stocks Trading Service



Complex Exercise 2

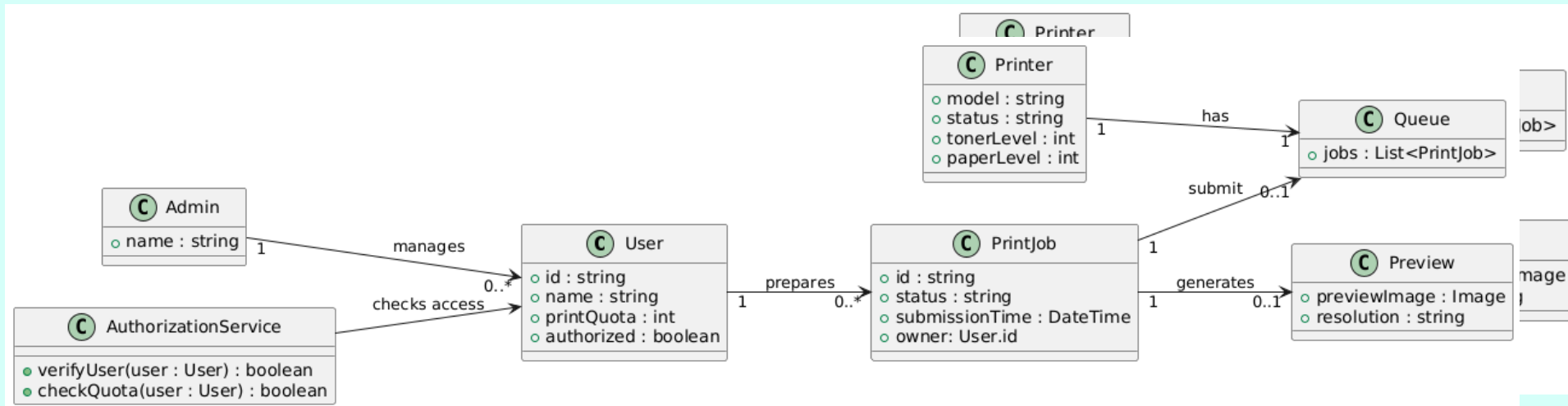
Network Printing Service

- There is a super high-resolution colour laser printer available on the network for users to print documents to. The service allows users to preview the output of their document on their screens. In addition, the user can also view the status of the printer to see whether there are other documents waiting to be printed and whether there are any problems with the printer (such as paper jams, out of paper, low on toner, etc...). In addition, users can monitor their own print jobs and delay or delete jobs as they see fit.
- To use this service, a user needs to have the proper authorization and print quota to print. A system administrator manages users and their print quotas.



Complex Exercise 2 solution

Network Printing Service





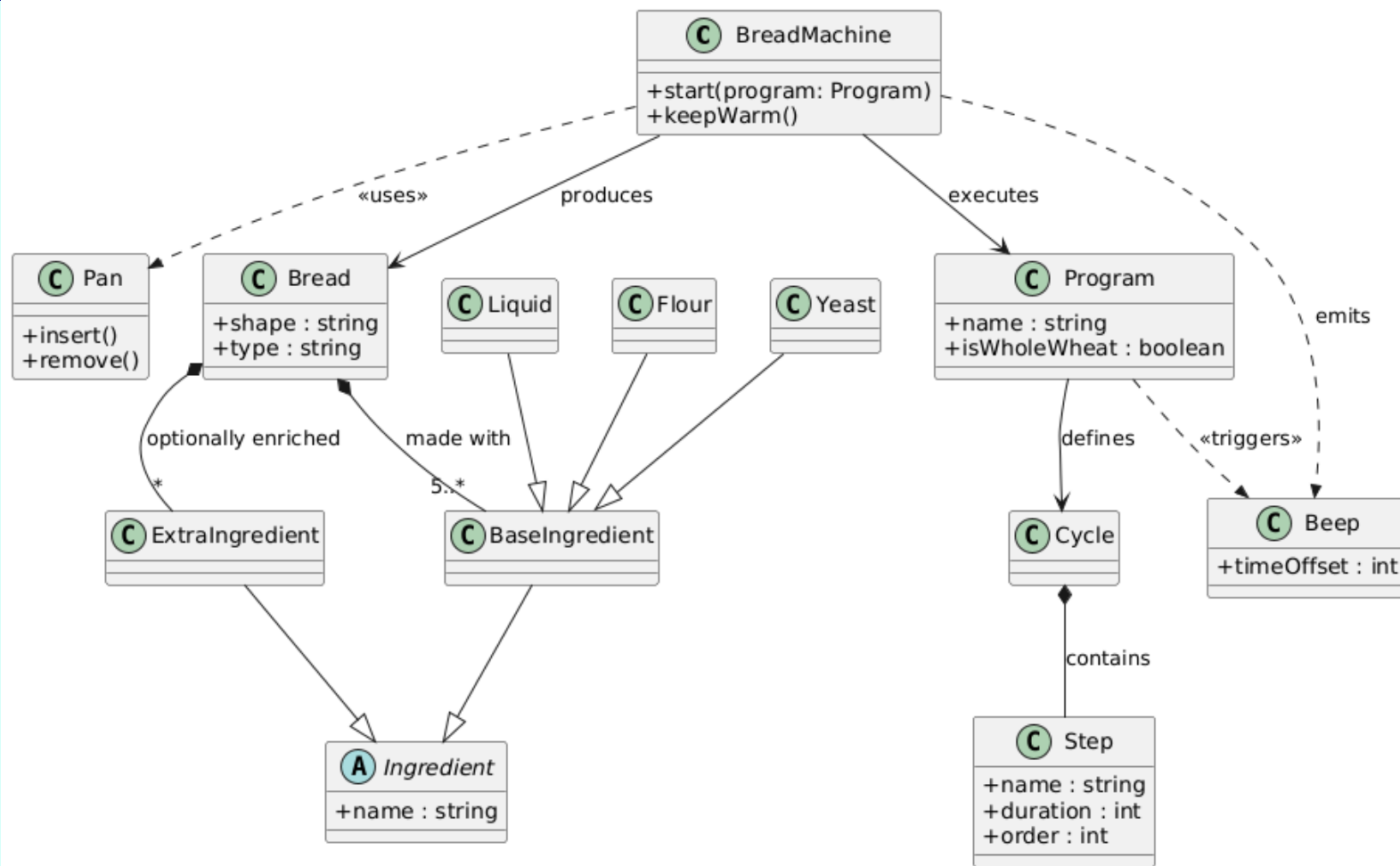
Complex Exercise 3

Bread Machine (Class & UC)

- The bread machine allows the preparation of delicious loaves, either round or “box-shaped.” Using the machine is simple: remove the pan, add the ingredients, reinsert the pan, select the program, and start the machine.
- At the end of baking, the machine keeps the bread warm for an hour. During this hour, the bread must be removed and allowed to cool, otherwise it hardens. Before slicing, one must wait another 30 minutes.
- All loaves are made with the same basic ingredients:
- Liquid (water or milk), flour (white or whole wheat), salt, sugar, and yeast (fresh or dry). Additionally, loaves can be enriched with extra ingredients (added after a beep): olives, seeds, or sun-dried tomatoes.
- All loaves follow the same baking cycle, except whole wheat bread, which requires a preheating phase, kneading, resting twice (for rising), and baking..

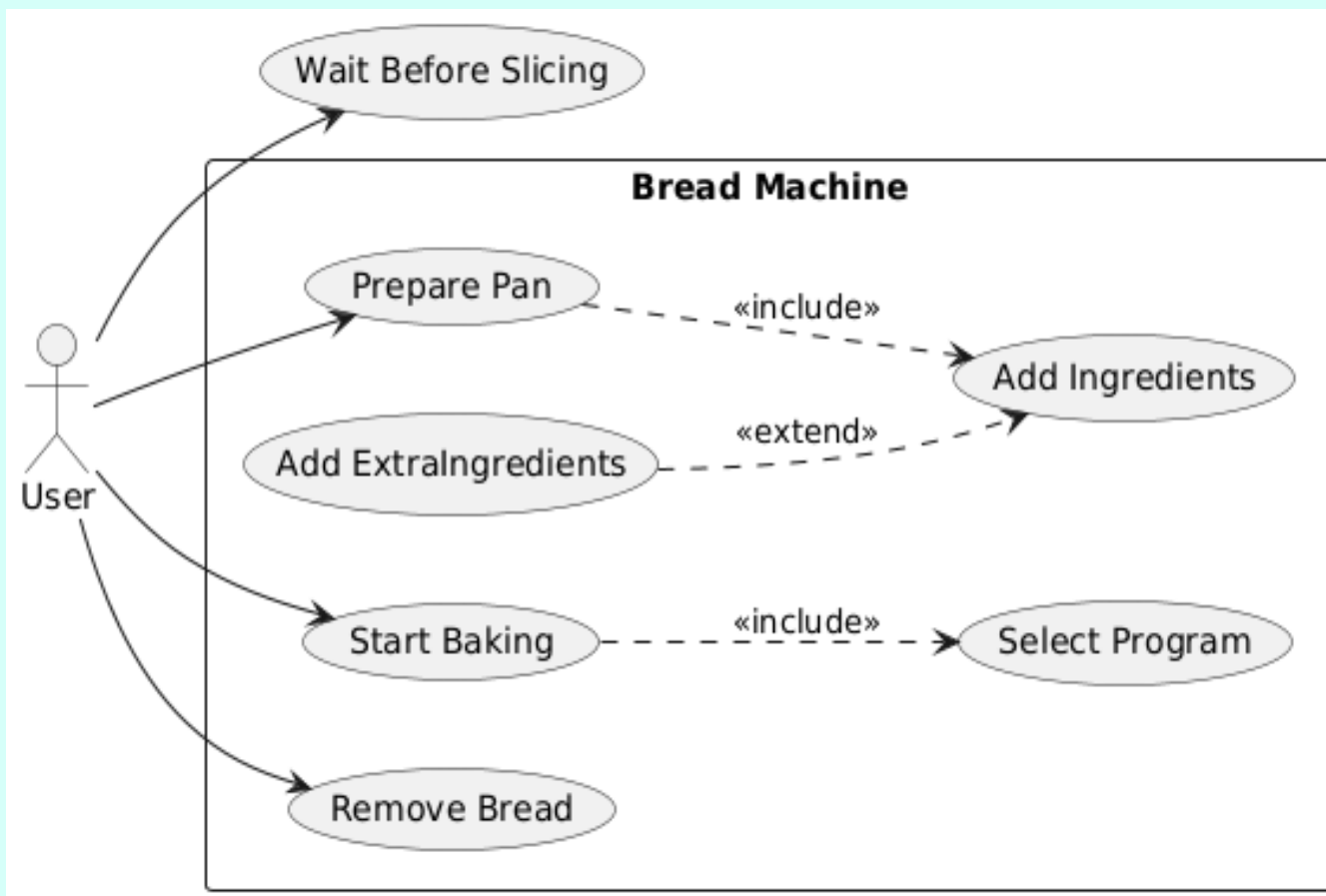
Complex Exercise 3 solution

Bread Machine



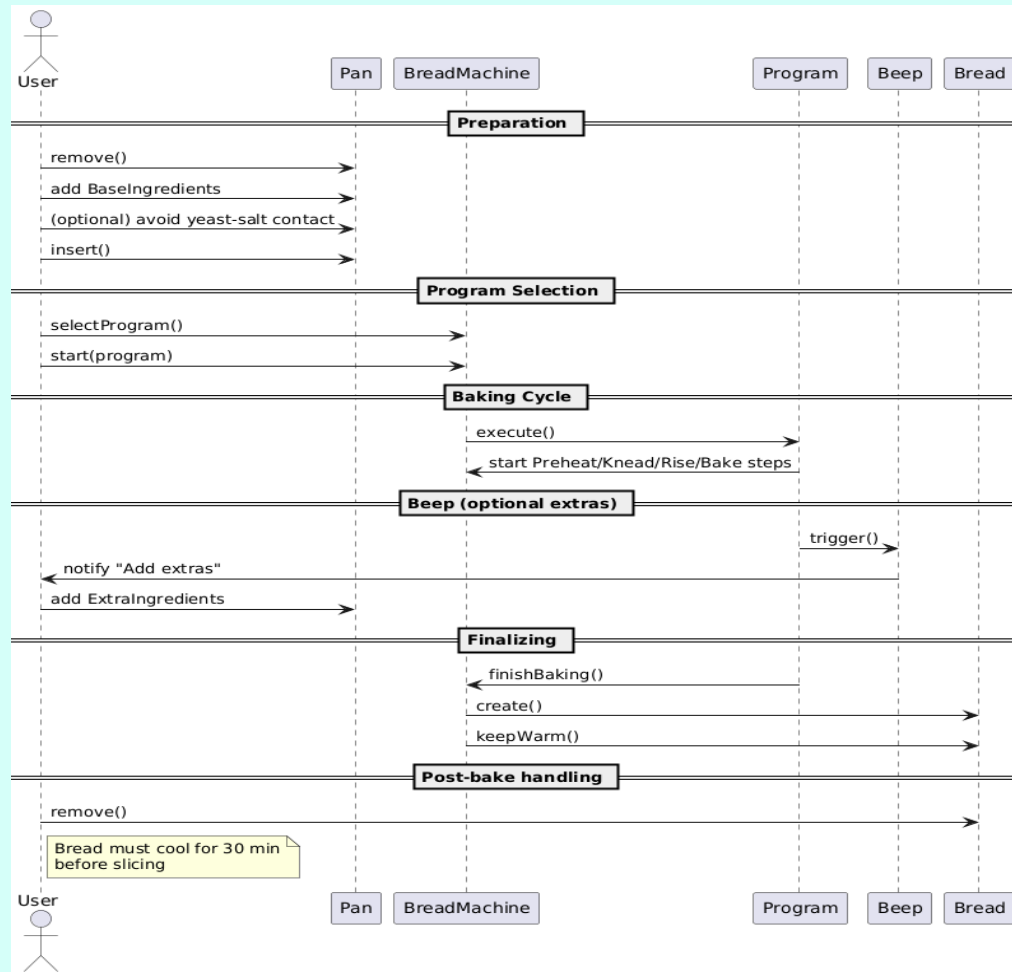
Complex Exercise 3 solution

Bread Machine use case diagram



Complex Exercise 3 solution

Bread Machine sequence preview





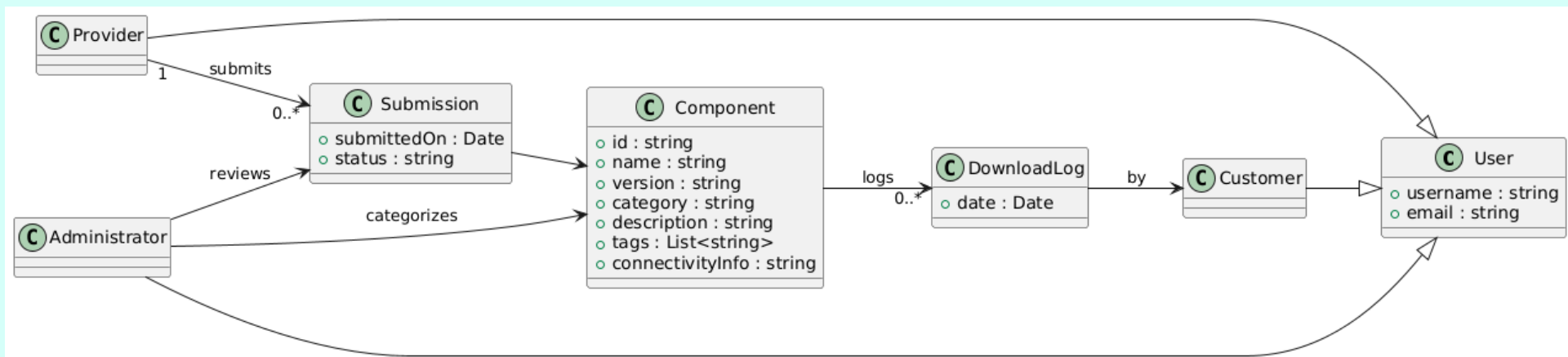
Complex Exercise 4

Component Brokerage System

- This system essentially acts as a broker for software components. When developers have completed development of their software, they can deploy them as reusable software components for others to use. By connecting to the system over the Internet, these developers can submit their components to the system. An administrator then reviews the component for its functionality and ways of connecting to other components, categorizes it, and publishes it in a publicly-viewable area.
- Customers (such as other developers) can then connect to the public system and browse/search the components. When they have found something useful, they can download it for use on their own machine.
- Later, the providers of the components can connect to the system and view the download statistics of their components. They can also add/remove components from the system.

Complex Exercise 4 Solution

Component Brokerage System





Complex Exercise 4

Component Brokerage System

- This system essentially acts as a broker for software components. When developers have completed development of their software, they can deploy them as reusable software components for others to use. By connecting to the system over the Internet, these developers can submit their components to the system. An administrator then reviews the component for its functionality and ways of connecting to other components, categorizes it, and publishes it in a publicly-viewable area.
- Customers (such as other developers) can then connect to the public system and browse/search the components. When they have found something useful, they can download it for use on their own machine.
- Later, the providers of the components can connect to the system and view the download statistics of their components. They can also add/remove components from the system.



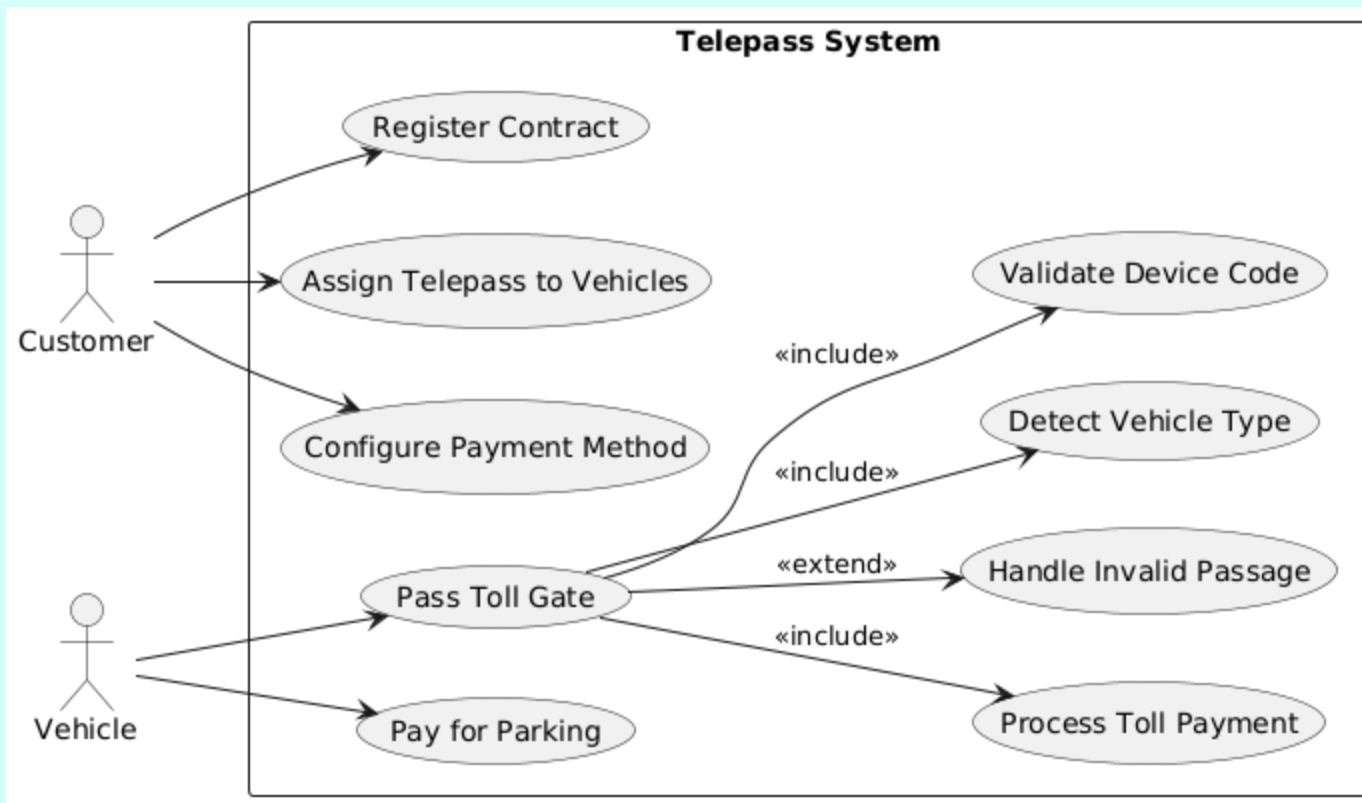
Complex Exercise 5

Telepass (1 of 2)

- The system involves the use of a device mounted on the windshield of vehicles, allowing toll payments without stopping at toll booths. In some locations, Telepass can also be used to pay for parking.
- The service is intended for individual persons (not companies).
- Each Telepass device is assigned to a customer through a contract, which defines the payment method — consisting of a periodic charge to an account, which may be a bank account, BancoPosta, or an enabled credit card.
- The Telepass device is identified by a unique code, and can be associated with up to three vehicles, whose license plates must be registered with Autostrade per l'Italia

Complex Exercise 5

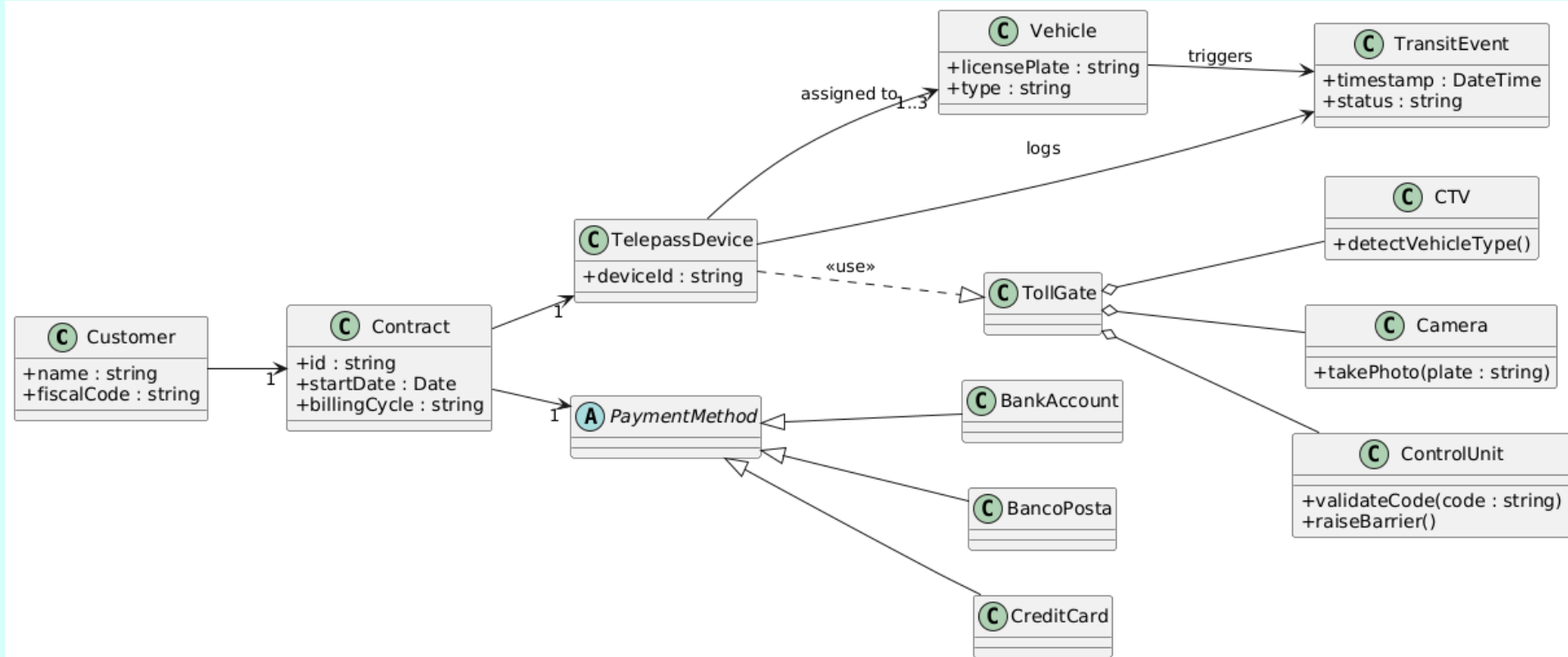
Telepass solution





Complex Exercise 5

Telepass solution





Complex Exercise 5

Telepass (2 of 2)

- The practical operation of the Telepass service is (relatively) simple: when a vehicle approaches a toll gate, an optical system (CTV) recognizes the type of vehicle and triggers the gate to emit a signal requesting identification.
- The Telepass device responds to the signal from the ground transmitter by retransmitting a unique identification code.
- After half a second, the ground control unit issues the command to raise the barrier, provided it has received a valid code from the device. If not, a special camera photographs the license plate to trace the person responsible for the irregular transit.
- In any case, a second optical system confirms the vehicle classification.

Propose an Use Case diagram and a Class Diagram



Further Proposed Exercise

Bug Tracking System

- Developers use this system to track bugs in an on-going software project. Developers who find bugs can submit a report. Other developers can then assign the bug to a particular developer (especially the developer responsible for the software module) to fix it. In addition, users can browse/search all the bugs in the system so far.
- An administrator manages users to restrict access to the bug tracking system. In addition, the administrator should also be able to generate reports on the state of the project in the form of a set of web pages updated daily at 2am.